

Real User Monitoring: Session Replay & On-Demand Session Replay

How Splunk Observability Cloud captures, replays, and protects real user sessions

What Is Real User Monitoring (RUM)?

Front-end visibility into what real users actually experience

RUM captures how your application performs from the browser or mobile device of a real user — page loads, clicks, errors, and network calls — as they happen in production.

Frontend Focus

Measures the experience in the browser/app, not just backend response time

Errors & Crashes

Captures JavaScript errors, crashes, and slow resource loads as they happen

Connected to APM

Correlates with backend spans for a full front-to-back view of a request

What Is Session Replay?

A video-like reconstruction of what one user actually saw and did

- Reconstructs a user's session as a visual playback — page views, clicks, scrolls, and navigation, in the order they happened
- Built from a lightweight DOM snapshot recording, not an actual screen-capture video — this keeps data volume low
- Synchronized with the RUM session waterfall, so you can see the exact moment a JavaScript error or slow request occurred
- Works for both browser and mobile applications (iOS, Android, Flutter, React Native)
- Maximum replay duration is 4 hours per session

Why it matters

- Cuts troubleshooting time — see the problem instead of guessing from a description
- Shows real UX friction, not just error counts
- Lets teams verify a fix by replaying the same journey

Continuous Recording vs. On-Demand Session Replay

Two ways to control when a session gets recorded

CONTINUOUS RECORDING

- Recording starts automatically for every eligible session
- Governed by a sample rate — e.g., record 10% of all sessions
- Good for broad UX visibility across your whole user base
- Uses more replay quota, since many sessions are captured whether or not they contain a problem

ON-DEMAND SESSION REPLAY

- Recording is started and stopped in code, using the sessionReplay SDK's start/stop methods
- Triggered only when it matters — e.g., after an error, a specific user action, or a support request
- Recording quality can also be adjusted at that moment
- Uses replay quota far more efficiently — you capture the sessions you actually need

How On-Demand Session Replay Works

From a trigger in your code to a playable session



A recording ends when the code calls `sessionReplay.stop()`, or automatically after the max session duration is reached.

The same SDK also lets you control recording quality at the moment you start a session — for example, capturing at higher fidelity for a critical checkout flow and lower fidelity elsewhere to save storage.

Privacy Is Built In

Session Replay is designed to be private by default

Text redaction by default

Text on the page is masked automatically unless explicitly allowed

Image redaction available

Images can be excluded or masked to hide sensitive visual content

Obfuscated before it leaves the browser

Redaction happens at the instrumentation layer — PII never reaches Splunk in the raw

No PII by design

Emails, addresses, and other personal identifiers are not captured as part of the replay

Note: you are still responsible for providing notice and obtaining any consent required by law before collecting session data.

Summary

When to use which approach

Use continuous recording when

you want broad, ongoing visibility into UX across a sample of all users

Use on-demand session replay when

you only need to capture specific, high-value moments — errors, crashes, or flagged user actions

Either way, session replay

is correlated with the RUM waterfall and protected by default-on redaction